Service Charter Document

Fest Management Platform

Purpose

To automate and centralize the lifecycle of college hackathons and technical fests, replacing fragmented manual tools with a single, high-performance platform for MITAOE.

Objective

To make the process of organizing, registering for, and managing hackathons:

- **Time Saving:** Reducing the administrative workload for club coordinators by automating team formation and registration tracking.
- **Precise and Accurate Result:** Ensuring zero data duplication and accurate payment verification for paid events.
- **Well Structured and Managed:** Providing a clear hierarchy for college admins to oversee multiple clubs and fests from one dashboard.

Demand / Opportunity

There is a significant opportunity as, currently, no dedicated, all-in-one hackathon platform is tailored for the MITAOE ecosystem. Most developers and club leads are forced to use a mix of Google Forms, WhatsApp, and Excel, which fails when handling long and complex participant models and team-based relationships.

Business Requirements

- **BR1: User Authentication**

  Description:
  A secure authentication mechanism must be implemented to control access to the platform.

  Requirement:
  Users including **Admins, Coordinators, and Students** must log in using a valid **username/email and password** to access the platform. The system must verify credentials before granting access.

  Justification:
  This ensures **secure access control**, protects sensitive student data, and prevents unauthorized users from modifying event or platform settings.

- **BR2: Event / Hackathon Creation**

  Description:
  The system must allow organizers to create and configure events or hackathons easily.

  Requirement:
  Authorized users should be able to create an event by defining key attributes such as:

  - Event Name
  - Date and Time
  - Venue
  - Team Size
  - Eligibility Criteria
  - Event Category

  Justification:
  This functionality enables organizers to quickly deploy different types of events such as **technical fests, cultural programs, workshops, or hackathons**.

- **BR3: Relationship Mapping (Team Management)**

  Description:
  The system must support structured relationships between participants and teams.

  Requirement:
  The platform must allow mapping relationships between **Students, Teams, Mentors, and Projects**. Multiple students should be able to join a team.

  Justification:
  This automates complex **team formation and project allocation**, which is one of the most challenging aspects of managing hackathons and collaborative competitions.

- **BR4: Event Reusability**

  Description:
  The platform must allow organizers to reuse previous event configurations.

  Requirement:
  Existing event templates, registration structures, and configurations should be reusable for future events with necessary modifications.

  Justification:
  This reduces repetitive setup work and allows clubs to **replicate successful event structures efficiently** for future semesters.

Technical Requirements

To manage high-traffic college fests and complex nested hierarchies, the following technical standards are established:

- **System Scalability:**
  The architecture must support horizontal scaling to handle up to **500 concurrent users** during peak registration windows for major fests.

- **Multi-Tier Nested Hierarchy:**
  Core logic must support the following relational structures:

  - College → Fest → Club → Event

  - College → Club → Standalone Event

- **Access Control (RBAC):**
  Implementation of **Role-Based Access Control** for four distinct tiers:

  - Super Admin (University)

  - Admin (College)

  - Clud Admin (Club Leads)

  - User (Students)

- **Transaction Integrity:**
  Atomic transaction handling must be implemented to prevent **over-booking of limited-capacity workshops or competitions**.

- **Notification Engine:**
  Real-time delivery of schedule changes and registration confirmations through **asynchronous messaging and automated SMTP triggers**

Technological Requirements

We have evaluated various stacks to ensure long-term stability and performance.

Component	Selected Technology	Primary Reason
Frontend	React	Provides a component-based architecture that allows for highly interactive and reusable UI elements for fests.
Backend	Java with Spring Boot	Offers enterprise-grade security and superior multi-threading capabilities for handling thousands of simultaneous registrations.
Database	PostgreSQL	Ensures strict data integrity for the complex relationships between colleges, clubs, and events.

Frontend Framework Comparison

Feature	React (Chosen)	Angular	Vue.js
Learning Curve	Moderate	High	Low
Performance	High (Virtual DOM)	High (Real DOM)	High (Virtual DOM)
Ecosystem	Massive (Tailwind CSS, shadcn/ui)	Full-featured but rigid	Smaller than React
Decision	**React**	Not Selected	Not Selected

Database Postgres: Why PostgreSQL?

A rigorous comparison was conducted between SQL and NoSQL variants to determine the best fit for Unbound's relational data.

Feature	PostgreSQL (Selected)	MySQL	MongoDB
Data Model	Relational (Strict)	Relational (Strict)	Document (Flexible)
Joins	Excellent (Nested Data)	Good	Poor (Requires Lookups)
ACID Compliance	Full (High Integrity)	Full	Document-level only
Scalability	Vertical and Horizontal	Mostly Vertical	Excellent Horizontal
Nested Logic	JSONB support for flexible fields	Limited JSON support	Native JSON

PostgreSQL is the chosen database because the "Fest > Club > Event" relationship is inherently relational. PostgreSQL provides the strict data integrity needed for registrations while offering JSONB for any unstructured event metadata.

Human Resources

1. **1 Project Lead (Scrum Master):** Responsible for sprint planning, removing blockers, and stakeholder alignment.
2. **2 Frontend Developers:** Responsible for the Student Discovery Portal and the Club Organizer Dashboard using React.
3. **2 Backend Developers:** Responsible for Java API architecture, Database Schema design, and Security protocols.
4. **1 UI/UX Designer:** Responsible for mobile-first design, prototyping, and accessibility compliance.
5. **1 QA Engineer:** Responsible for automated unit testing (JUnit) and End-to-End (E2E) registration flow testing.

PESTEL Analysis:

Factor	Impact on Project	Example
Political	No direct political restrictions affecting the platform	MITAOE operates independently for internal event management
Economic	Reduces manual management cost for clubs	Less dependency on paid event tools
Social	Encourages student participation and engagement	Students easily discover events
Technological	-	-
Environmental	Digital platform reduces paperwork and manual forms	Eco-friendly event registration
Legal	Must comply with data protection and payment regulations	Secure handling of student data and payment transactions

Risk Analysis: FMEA-based model & ISO 31000

Formula used
RPN = Likelihood × Impact × Detectability

Where:

- **Likelihood (1–5)** = chance the risk occurs
- **Impact (1–5)** = severity if it happens
- **Detectability (1–5)** = how hard it is to detect before damage occurs

Risk Analysis Link :

https://docs.google.com/spreadsheets/d/1PWZpNunfXjQh3dDIHBo_Y4v-_pOvMltn0CBHHJmthol/edit?usp=sharing

Timeline / Milestone:

Phase	Milestone	Tasks	Timeline
1	Requirement Analysis & Planning	- Gather and Document project requirements - Define & construct functional flow of the application - Identify third-party services and dependencies - Design UI mockups and wireframes	Week 1
2	Database Design and API Development	- Design database schema - Set up database configurations - Develop API endpoints - Implement user authentication setup - Perform API testing using postman	Week 2
3	UI Development and API Integration	- Develop UI components and pages - Implement state management - Connect frontend with backend - Implement error handling & validation - Optimize API calls	Week 3
4	Testing & Debugging	- Perform UI/UX testing - Perform API integration testing - Fix bugs and optimize performance	Week 4
5	Deployment & Final Review	- Set up cloud hosting (AWS) - Implement CI/CD pipelines - Monitor and resolve deployment issues - Conduct final review and documentation	Week 4

Stakeholder Table :

Stakeholder	Role
MITAOE Admin	Oversees events and clubs
Club Coordinators	Create and manage events
Students	Register and participate in events
Development Team	Build and maintain the platform
QA Team	Ensure system quality

Work Breakdown Structure (WBS)

1.0 Requirement Analysis & System Design

1.1 Requirement Documentation

- 1.1.1 Create Project Charter Document
- 1.1.2 Create Business Requirements Document (BRD)
- 1.1.3 Create Software Requirements Specification (SRS) Document
- 1.1.4 Prepare project documentation

1.2 System Design

- 1.2.1 Define system architecture diagram
- 1.2.2 Create API design documentation
- 1.2.3 Define RBAC roles
- 1.2.4 Identify third-party integrations

1.3 Database Design

- 1.3.1 Create database ER diagram

1.4 UI Planning

- 1.4.1 Create UI wireframes

1.5 Project Setup

- 1.5.1 Setup GitHub repository structure
- 1.5.2 Setup branching strategy
- 1.5.3 Setup project board

2.0 Backend Core Development

2.1 Backend Initialization

- 2.1.1 Initialize Spring Boot backend project

2.2 Database Setup

- 2.2.1 Configure PostgreSQL database connection
- 2.2.2 Implement database schema

2.3 Authentication & Authorization

- 2.3.1 Create User authentication API JWT
- 2.3.2 Implement password hashing using BCrypt
- 2.3.3 Implement RBAC authorization

2.4 Core API Development

- 2.4.1 Create Club management APIs
- 2.4.2 Create Fest management APIs
- 2.4.3 Create Event CRUD APIs
- 2.4.4 Implement Event registration APIs

2.5 Backend Validation & Reliability

- 2.5.1 Implement registration capacity validation
- 2.5.2 Setup global exception handling

2.6 Backend Verification

- 2.6.1 Test APIs using Postman
- 2.6.2 Swagger API Documentation Setup

3.0 Frontend Development

3.1 Frontend Setup

- 3.1.1 Setup React project
- 3.1.2 Setup React Router

3.2 Authentication Interface

- 3.2.1 Implement authentication pages

3.3 Student Module

- 3.3.1 Implement Student Dashboard
- 3.3.2 Implement Event Listing Page
- 3.3.3 Implement Event Details Page
- 3.3.4 Implement My Events page

3.4 Club Module

- 3.4.1 Implement Club dashboard
- 3.4.2 Implement Create Edit Event pages

3.5 Frontend Integration

- 3.5.1 Connect frontend with backend APIs

3.6 Validation & UX Handling

- 3.6.1 Implement form validation
- 3.6.2 Implement loading states and error handling

4.0 Feature Completion & Payment Integration

4.1 Payment Integration

- 4.1.1 Integrate Razorpay payment gateway
- 4.1.2 Implement payment success and failure callbacks
- 4.1.3 Create payment history module

4.2 Administrative Features

- 4.2.1 Implement Admin dashboard
- 4.2.2 Implement club approval system

4.3 Registration Control

- 4.3.1 Implement event capacity locking

4.4 Notification & Reporting

- 4.4.1 Implement email notifications using SendGrid
- 4.4.2 Implement CSV export for registrations

4.5 Frontend Security & Enhancement

- 4.5.1 Improve UI responsiveness
- 4.5.2 Implement role based route protection

5.0 Testing Optimization & Deployment

5.1 Testing

- 5.1.1 Write JUnit tests for backend
- 5.1.2 Perform API integration testing
- 5.1.3 Test event registration workflow
- 5.1.4 Test payment workflow

5.2 Bug Resolution

- 5.2.1 Fix UI bugs
- 5.2.2 Fix backend bugs

5.3 Performance Optimization

- 5.3.1 Optimize database queries
- 5.3.2 Improve API response performance

5.4 Deployment

- 5.4.1 Setup Amazon Web Services EC2 instance
- 5.4.2 Setup PostgreSQL RDS
- 5.4.3 Deploy backend service
- 5.4.4 Deploy frontend build
- 5.4.5 Configure domain and SSL

5.5 Final Validation

- 5.5.1 Final production testing